SPI Protocol Implementation in Verilog HDL

Final Project Report

Design, Verification, Synthesis & Implementation of an SPI Slave with Single-Port RAM Interface

Omar Gamal Mohamed
Adel El Sayed Ibrahim Fouda
CTRL+S team

4.1 SPI_Slave.v

```
module SPI_Slave (
    input  wire       MOSI,
    input  wire       SS_n,
    input  wire       clk,
    input  wire       rst_n,
    input  wire       tx_valid,
    input  wire [7:0] tx_data,
    output reg        MISO,
    output reg        rx_valid,
    output reg  [9:0] rx_data
);

    localparam IDLE      = 3'b000;
    localparam CHK_CMD   = 3'b001;
    localparam WRITE     = 3'b010;
    localparam READ_ADD  = 3'b011;
    localparam READ_DATA = 3'b100;

(* fsm_encoding = "gray" *)
    reg [2:0] cs, ns;
    reg [4:0] bit_counter;
    reg       rd_addr_received;
    reg [7:0] tx_data_reg;

    always @(posedge clk ) begin
        if (!rst_n)
            cs <= IDLE;
        else
            cs <= ns;
    end

    always @(*) begin
        case (cs)
            IDLE: begin
                if (!SS_n)
                    ns = CHK_CMD;
                else
                    ns = IDLE;
            end

            CHK_CMD: begin
                if (SS_n)
                    ns = IDLE;
                else if (MOSI == 1'b0)
                    ns = WRITE;
                else begin
                    if (!rd_addr_received)
                        ns = READ_ADD;
                    else
                        ns = READ_DATA;
                end
            end

            WRITE: begin
                if (SS_n) ns = IDLE;
                else      ns = WRITE;
            end
```

```
        READ_ADD: begin
            if (SS_n) ns = IDLE;
            else      ns = READ_ADD;
        end

        READ_DATA: begin
            if (SS_n) ns = IDLE;
            else      ns = READ_DATA;
        end

        default: ns = IDLE;
    endcase
end

always @(posedge clk ) begin
    if (!rst_n) begin
        rx_data          <= 10'd0;
        rx_valid         <= 1'b0;
        MISO             <= 1'b0;
        bit_counter      <= 5'd0;
        rd_addr_received <= 1'b0;
    end else begin
        rx_valid <= 1'b0;

        case (cs)
            IDLE: begin
                bit_counter <= 5'd0;
                MISO        <= 1'b0;
                rx_data     <= 10'd0;
            end

            CHK_CMD: begin
                bit_counter <= 5'd0;
            end

            WRITE: begin
                rx_data <= {rx_data[8:0], MOSI};
                if (bit_counter == 5'd9) begin
                    rx_valid    <= 1'b1;
                    bit_counter <= 5'd0;
                end else begin
                    bit_counter <= bit_counter + 1'b1;
                end
            end

            READ_ADD: begin
                rx_data <= {rx_data[8:0], MOSI};
                if (bit_counter == 5'd9) begin
                    rx_valid         <= 1'b1;
                    bit_counter      <= 5'd0;
                    rd_addr_received <= 1'b1;
                end else begin
                    bit_counter <= bit_counter + 1'b1;
                end
            end

           READ_DATA: begin
```

```
    if (bit_counter <= 5'd9) begin
        rx_data <= {rx_data[8:0], MOSI};
        if (bit_counter == 5'd9) begin
            rx_valid <= 1'b1;
        end
        bit_counter <= bit_counter + 1'b1;
    end

    else if (bit_counter == 5'd10) begin

        MISO        <= tx_data[7];
        bit_counter <= bit_counter + 1'b1;
    end

    else if (bit_counter >= 5'd11 && bit_counter <= 5'd17) begin
        MISO        <= tx_data[5'd17 - bit_counter];
        bit_counter <= bit_counter + 1'b1;
    end

    else if (bit_counter == 5'd18) begin
        MISO             <= 1'b0;
        rd_addr_received <= 0;
        bit_counter      <= 5'd0;
    end
end
            endcase
        end
    end

endmodule
```

4.2 spr.v (Single-Port RAM)

```
module spr ( din ,rx_valid , clk , rst_n , dout , tx_valid  ) ;

parameter  MEM_DEPTH = 256 ,
           ADDR_SIZE = 8   ;

input  [9:0] din ;
input  clk , rst_n , rx_valid ;

output reg tx_valid ;
output reg [7:0] dout ;

reg [7:0] mem [MEM_DEPTH-1:0] ;
reg [ADDR_SIZE-1:0] rd_addr,wr_addr ;

always @(posedge clk) begin
    if (!rst_n)begin
        tx_valid <= 0 ;
        dout <= 0 ;
    end
    else begin
      tx_valid <= 1'b0;
        case (din[9:8])
            2'b00 : if (rx_valid) wr_addr <= din[7:0] ;
```

```
            2'b01 : if (rx_valid) mem[wr_addr] <= din[7:0] ;
            2'b10 : if (rx_valid) rd_addr <= din[7:0] ;
            2'b11 : begin if(rx_valid) begin  dout <= mem[rd_addr] ; tx_valid <= 1 ; end end
    endcase
    end

end
endmodule
```

4.3 SPI_Wrapper.v (Top Module)

```
module SPI_Wrapper (
    input  wire MOSI,
    input  wire SS_n,
    input  wire clk,
    input  wire rst,
    output wire MISO
);

    wire [9:0] rx_data;
    wire       rx_valid;
    wire [7:0] tx_data;
    wire       tx_valid;

    SPI_Slave spi_slave (
        .MOSI(MOSI),
        .SS_n(SS_n),
        .clk(clk),
        .rst_n(rst),
        .tx_data(tx_data),
        .tx_valid(tx_valid),
        .MISO(MISO),
        .rx_data(rx_data),
        .rx_valid(rx_valid)
    );

    spr ram (
        .din(rx_data),
        .rx_valid(rx_valid),
        .clk(clk),
        .rst_n(rst),
        .dout(tx_data),
        .tx_valid(tx_valid)
    );

endmodule
```

4.4 SPI_tb.v (Testbench)

```
`timescale 1ns/1ps

module SPI_tb ;

reg  MOSI, SS_n, clk, rst;
wire MISO ;

SPI_Wrapper DUT (
    .MOSI(MOSI),
```

```
    .SS_n(SS_n),
    .clk(clk),
    .rst(rst),
    .MISO(MISO)
);

initial begin
    clk = 0 ;
    forever begin
        #5 clk = ~clk ;
    end
end

initial begin

    MOSI = 0 ;
    rst  = 0 ;
    SS_n = 1 ;

    $readmemb("mem1.txt", DUT.ram.mem) ;

    @(negedge clk) ;
    rst  = 1 ;
    SS_n = 0 ;

    @(negedge clk) ;
    MOSI = 0 ;

    repeat (2) begin
        @(negedge clk);
        MOSI = 0 ;
    end

    repeat(8) begin
        @(negedge clk) ;
        MOSI = $random  ;
    end
    SS_n = 1 ;

    @(negedge clk) ;
    SS_n = 0 ;

    @(negedge clk) ;
    MOSI = 0 ;

    @(negedge clk) ;
    MOSI = 0 ;

    @(negedge clk) ;
    MOSI = 1 ;

    repeat(8) begin
        @(negedge clk) ;
        MOSI = $random  ;
    end
    SS_n = 1 ;
```

```
    @(negedge clk) ;
    SS_n = 0 ;

    @(negedge clk) ;
    MOSI = 1 ;

    @(negedge clk) ;
    MOSI = 1 ;

    @(negedge clk) ;
    MOSI = 0 ;

    repeat(8) begin
        @(negedge clk) ;
        MOSI = $random  ;
    end
    SS_n = 1 ;

    @(negedge clk) ;
    SS_n = 0 ;

    @(negedge clk) ;
    MOSI = 1 ;

    @(negedge clk) ;
    MOSI = 1 ;

    @(negedge clk) ;
    MOSI = 1 ;

    repeat(8) begin
        @(negedge clk) ;
        MOSI = $random  ;
    end

     repeat (8) @(negedge clk) ;

    SS_n = 1 ;

end

endmodule
```

5. Functional Verification (QuestaSim)

5.1 Full Transaction Waveform

The waveform below spans all four transactions in sequence. rx_data captures each shifted-in address/data frame (visible as the value changes on SS_n toggling), rx_valid pulses once per completed write/address frame, tx_data holds the byte read back from RAM, and tx_valid pulses when that byte is ready to be shifted out on MISO during the read-data transaction.

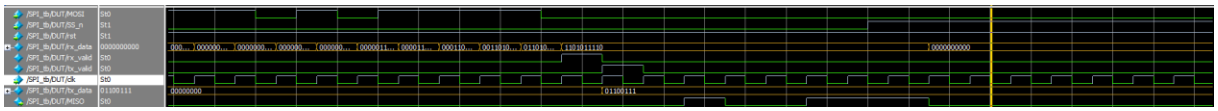


Figure 5.1 — QuestaSim waveform covering Write Address, Write Data, Read Address, and Read Data transactions

6. Linting (QuestaLint)

The RTL was checked with QuestaLint to catch coding-style, combinational-loop, and synthesis-mismatch issues prior to synthesis. The design achieved a Quality Score of 99.1%, with Nomenclature Style, Simulation, and Implementation readiness all at 100% and RTL Design Style at 98.9%. No lint errors were reported; one open Warning was flagged (multi_ports_in_single_line, STARC rule 3.5.6.3) in the spr module, noting that multiple ports were declared on a single line — a style issue only, with no functional or timing impact.

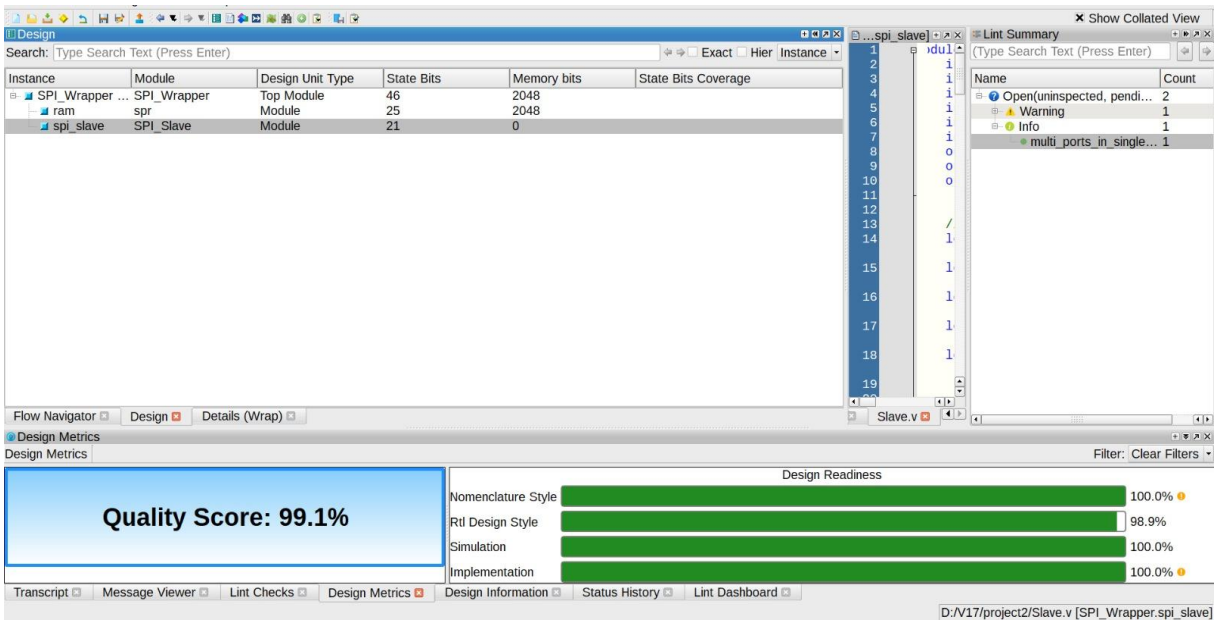


Figure 6.1 — QuestaLint Design Metrics: Quality Score 99.1%, Lint Summary (1 Warning, 1 Info)

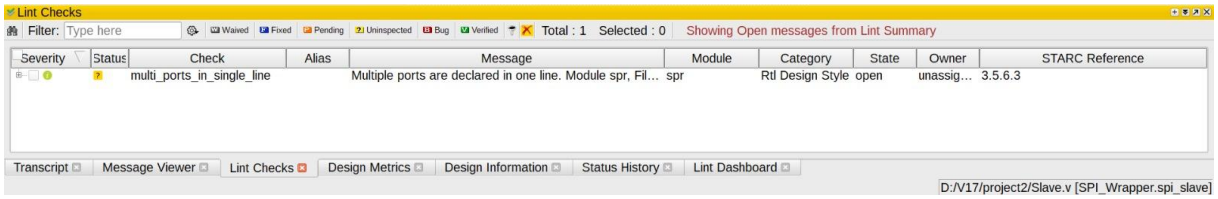


Figure 6.2 — Lint Checks detail: multi_ports_in_single_line warning in module spr

7. Elaborated Design (Top Level)

Before synthesis, Vivado's elaborated RTL schematic confirms the top-level connectivity: SPI_Wrapper exposes the external SPI interface (MOSI, SS_n, clk, rst, MISO) and internally wires spi_slave (SPI_Slave) to ram (spr) via rx_data, rx_valid, tx_data, and tx_valid.

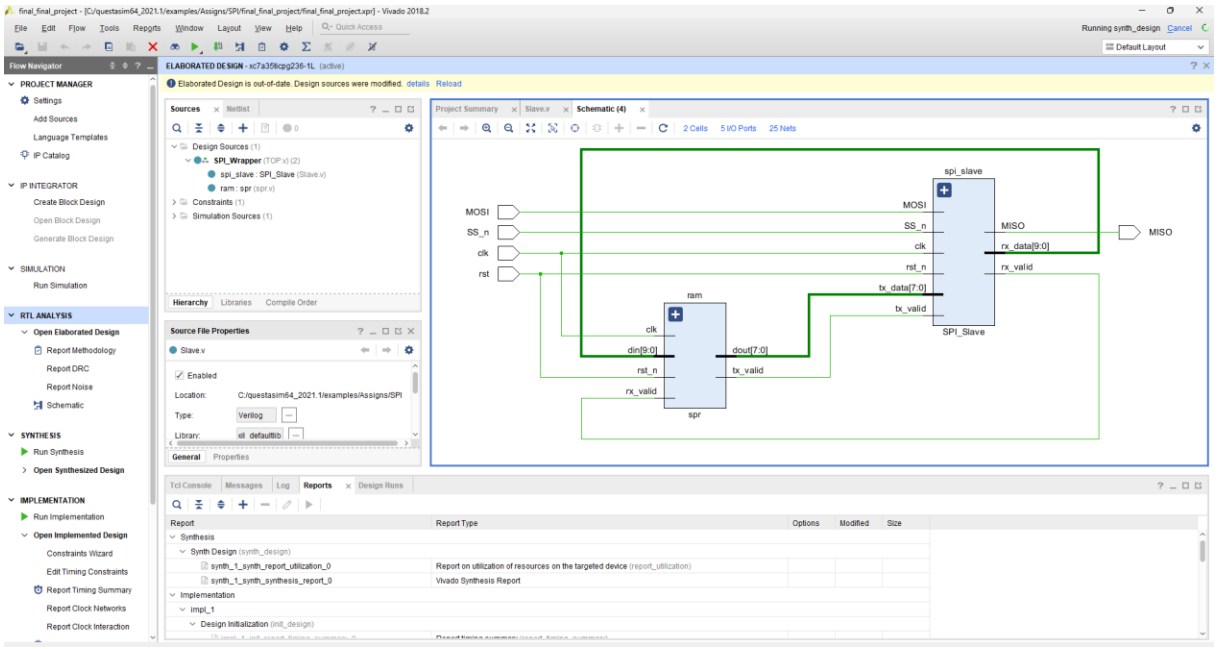


Figure 7.1 — Elaborated RTL schematic of SPI_Wrapper, showing spi_slave and ram sub-modules and the external SPI interface

8. Encoding: One-hot

This chapter groups the complete Vivado results — synthesis followed by implementation — for the FSM state register (cs_reg) synthesized with one-hot encoding.

8.1 Synthesis

8.1.1 Utilization

SPI_Wrapper synthesizes to 18 cells, 5 I/O ports, and 103 nets, comprising the spi_slave (SPI_Slave) and ram (spr) sub-modules plus debug-hub/ILA infrastructure.

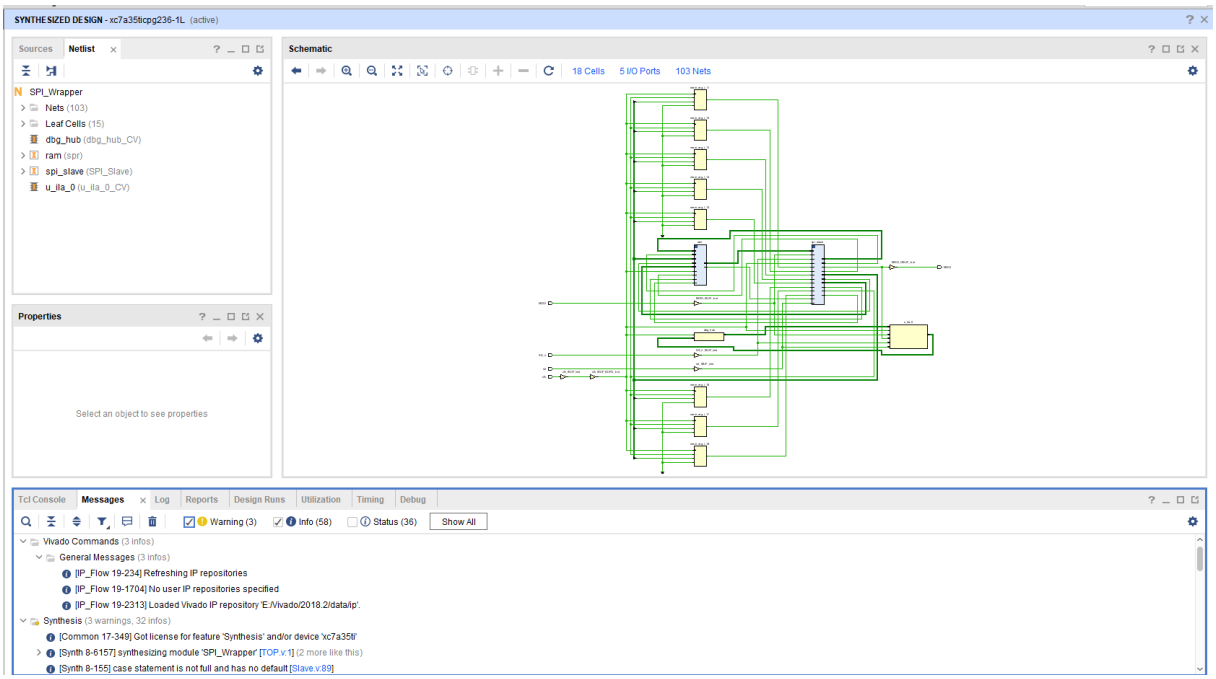


Figure 8.1 — Post-synthesis schematic of SPI_Wrapper, with synthesis Messages (3 Warnings, 58 Info)

8.1.2 Synthesis Report (Encoding Used)

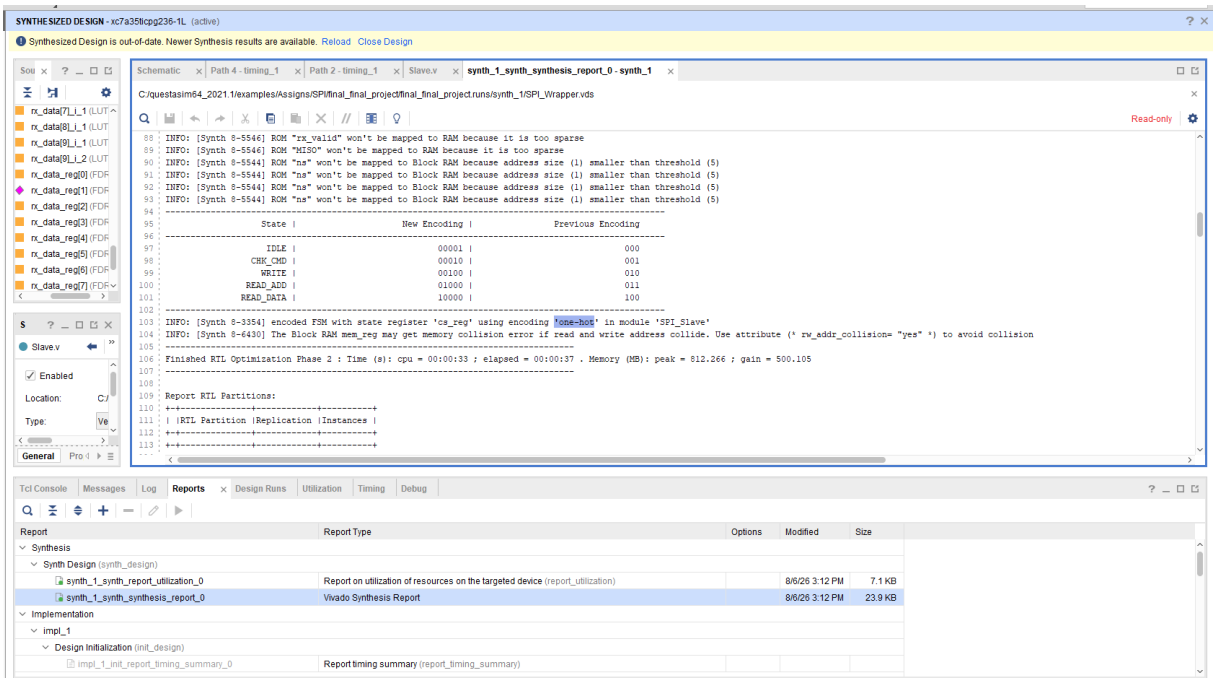


Figure 8.2 — Synthesis report: FSM state register 'cs_reg' encoded with 'one-hot' in module SPI_Slave

8.1.3 Timing Report

Post-synthesis timing closure is met with a Worst Negative Slack (WNS) of 6.657 ns, Worst Hold Slack (WHS) of 0.146 ns, and zero failing endpoints across setup, hold, and pulse-width checks (99 endpoints for setup/hold, 42 for pulse width). All user-specified timing constraints are met.

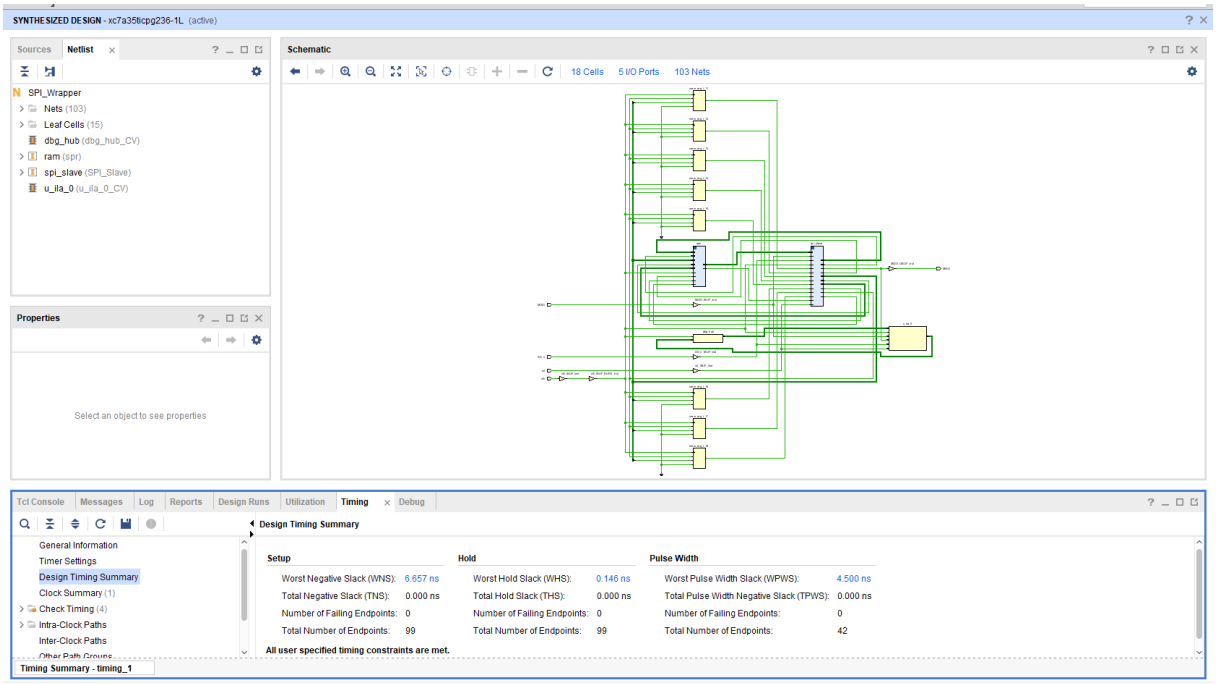


Figure 8.3 — Post-synthesis Design Timing Summary: WNS 6.657 ns, 0 failing endpoints

8.1.4 Critical Path

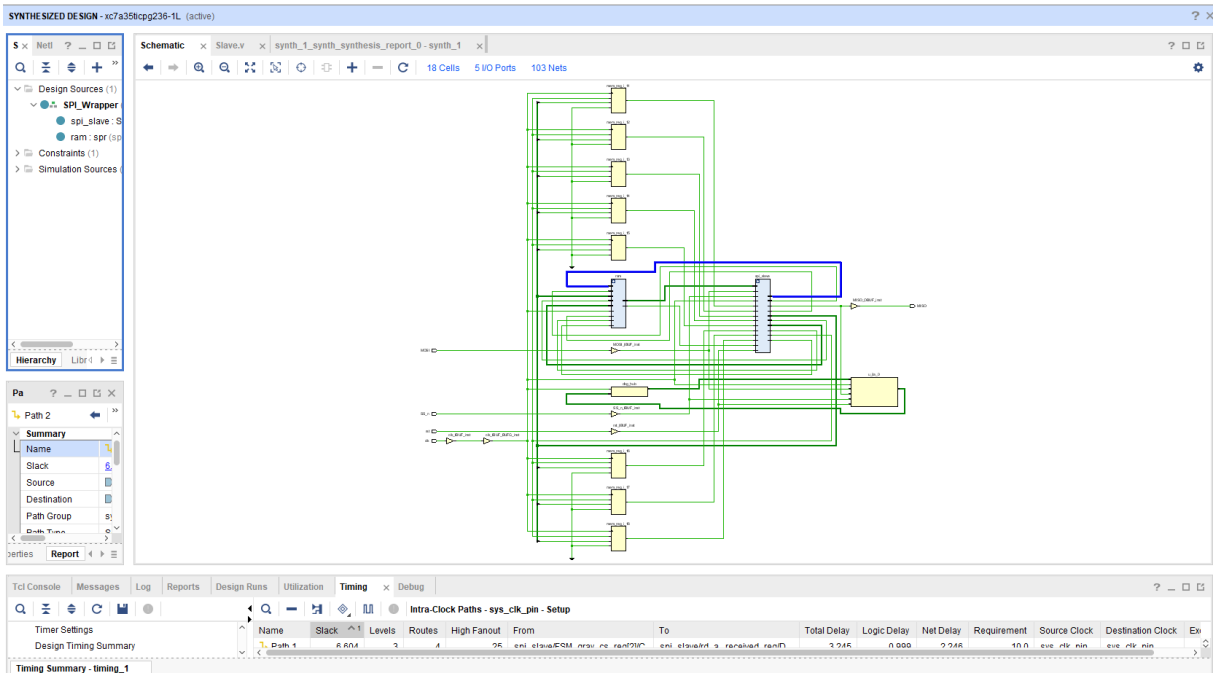


Figure 8.4 — Schematic with the critical path highlighted (Gray-encoded run, shown for reference), and the Intra-Clock Paths setup timing report for Path 2 (slack 6.604 ns)

8.1.5 Post-Synthesis Utilization

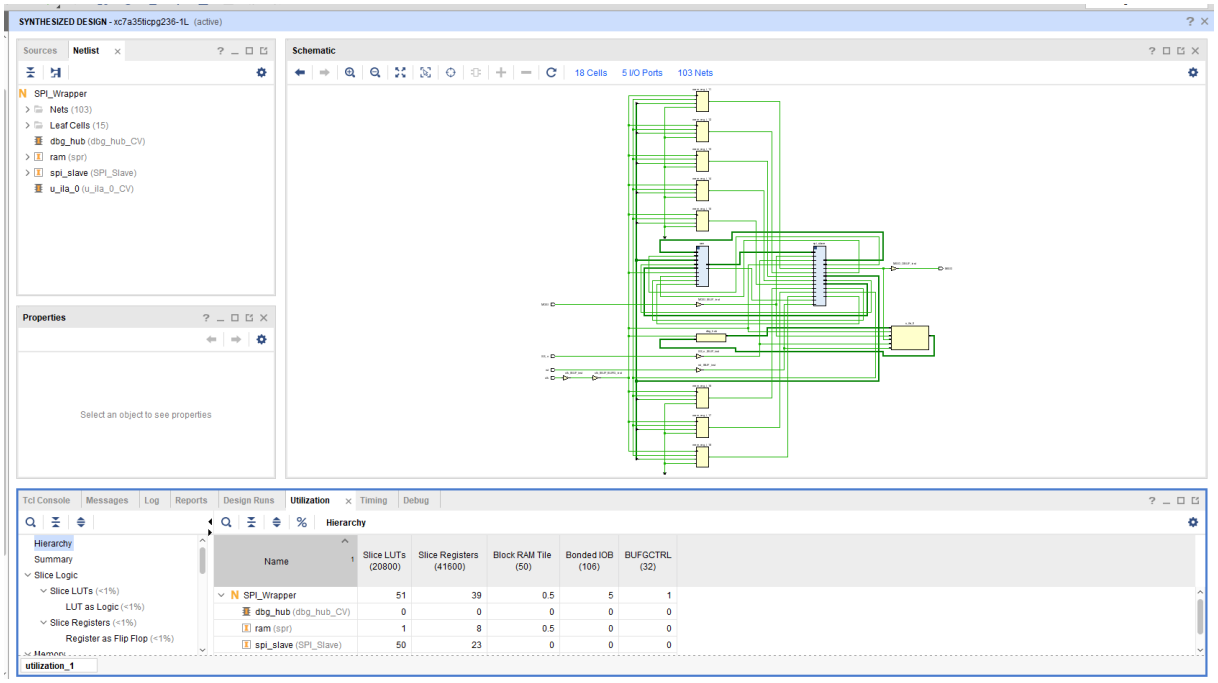


Figure 8.5 — Preliminary post-synthesis utilization by hierarchy

8.2 Implementation

8.2.1 Utilization Report

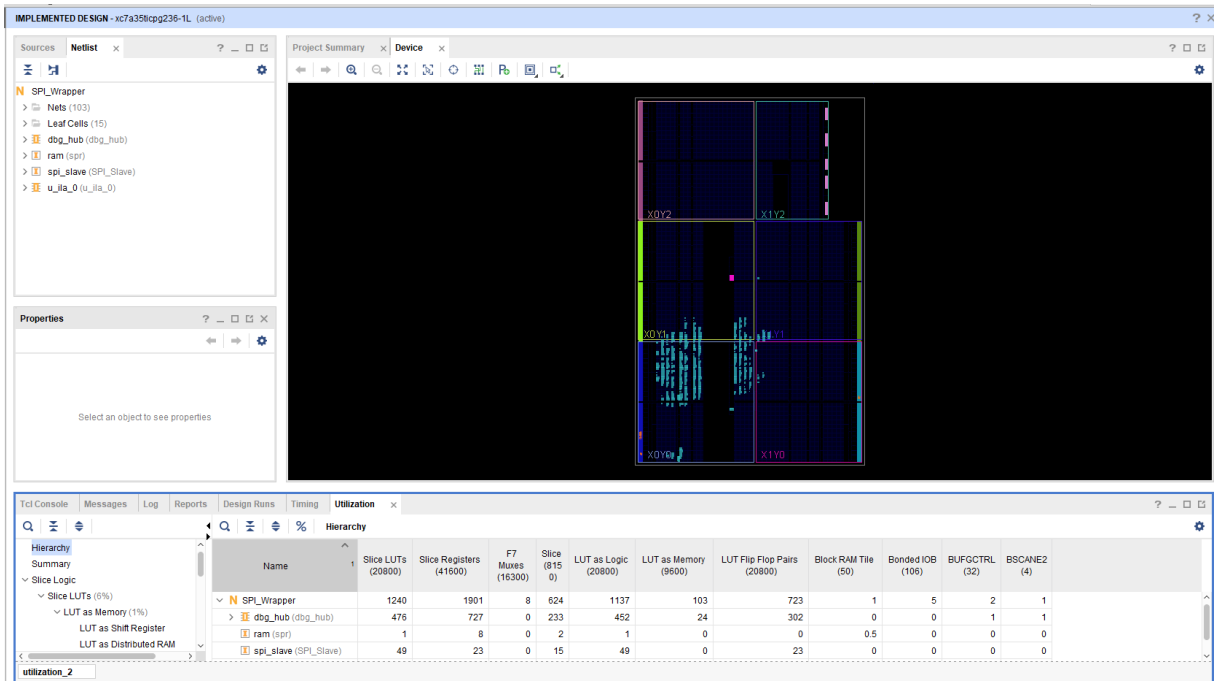


Figure 8.6 — Implemented design device view with post-implementation Utilization by hierarchy

8.2.2 Timing Report

Post-implementation timing closure is met with a Worst Negative Slack (WNS) of 4.215 ns, Worst Hold Slack (WHS) of 0.028 ns, Worst Pulse Width Slack (WPWS) of 3.750 ns, and zero failing endpoints across setup (3789 endpoints), hold (3773 endpoints), and pulse-width (2078 endpoints) checks. All user-specified timing constraints are met.

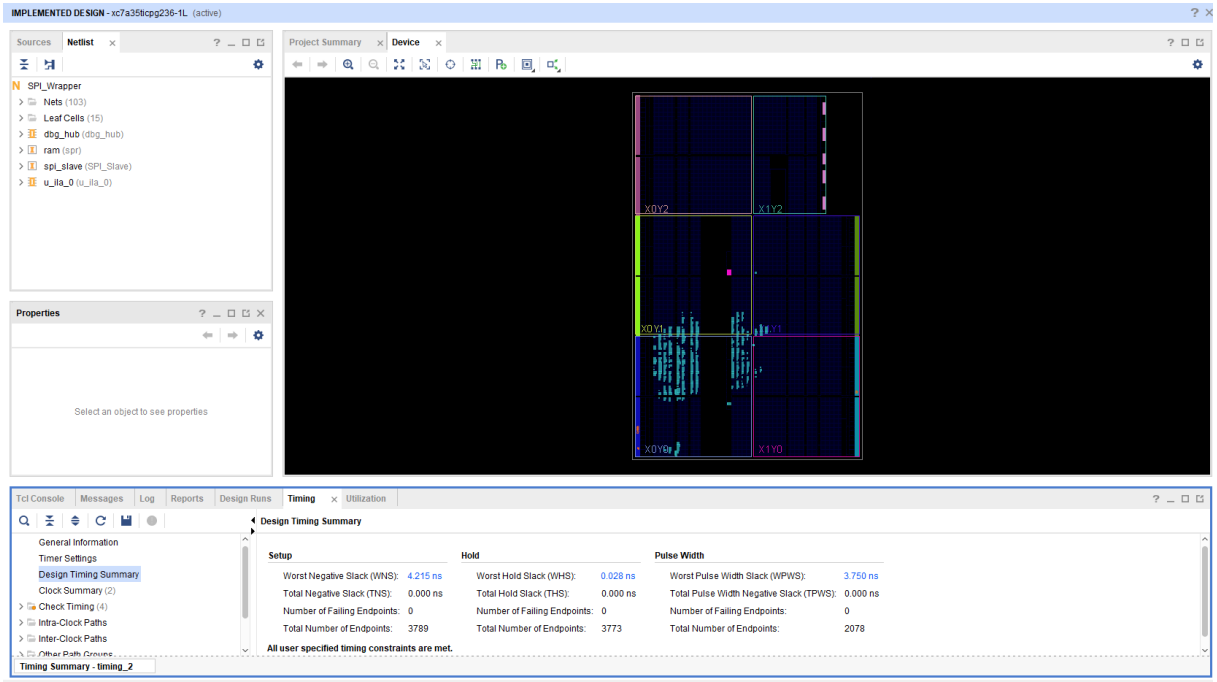


Figure 8.7 — Implemented design device view with post-implementation Design Timing Summary: WNS 4.215 ns, 0 failing endpoints

8.2.3 messages

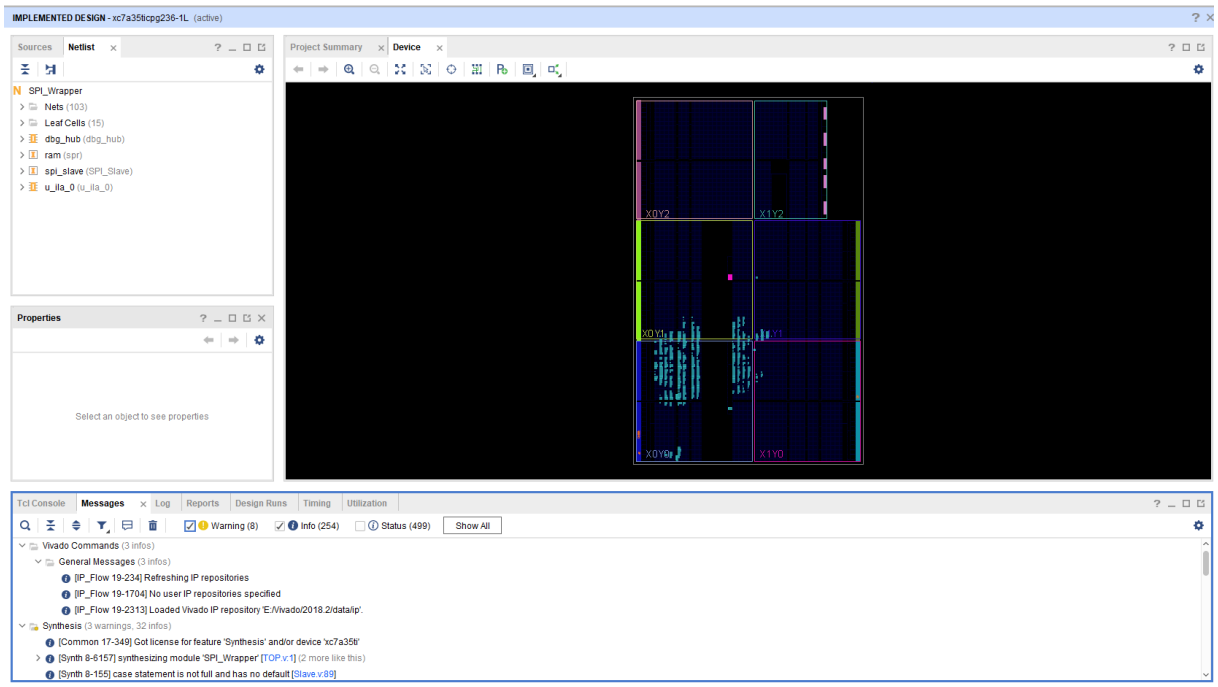


Figure 8.8 — Implemented design device (placement) view on xc7a35ticpg236-1L

9. Encoding: Gray

This chapter groups the complete Vivado results — synthesis followed by implementation — for the FSM state register (cs_reg) synthesized with Gray encoding.

9.1 Synthesis

9.1.1 Utilization

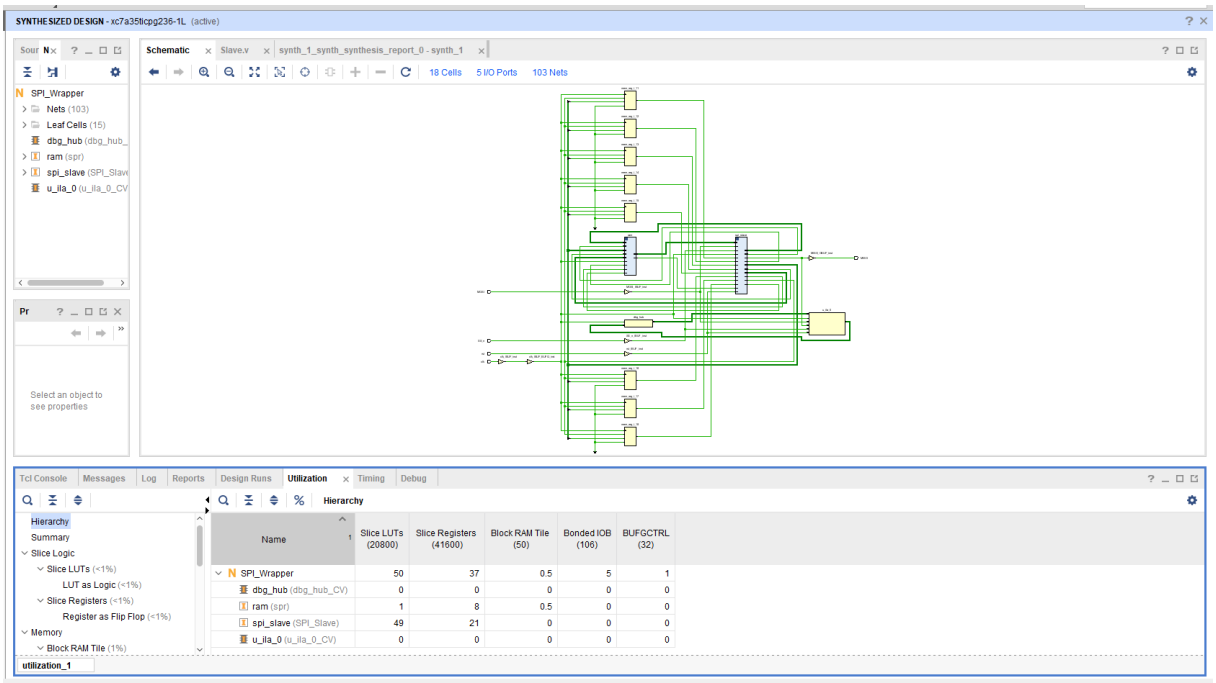


Figure 9.1 — Post-synthesis schematic of SPI_Wrapper for the Gray-encoded run, with post-synthesis Utilization by hierarchy

9.1.2 Synthesis Report (Encoding Used)

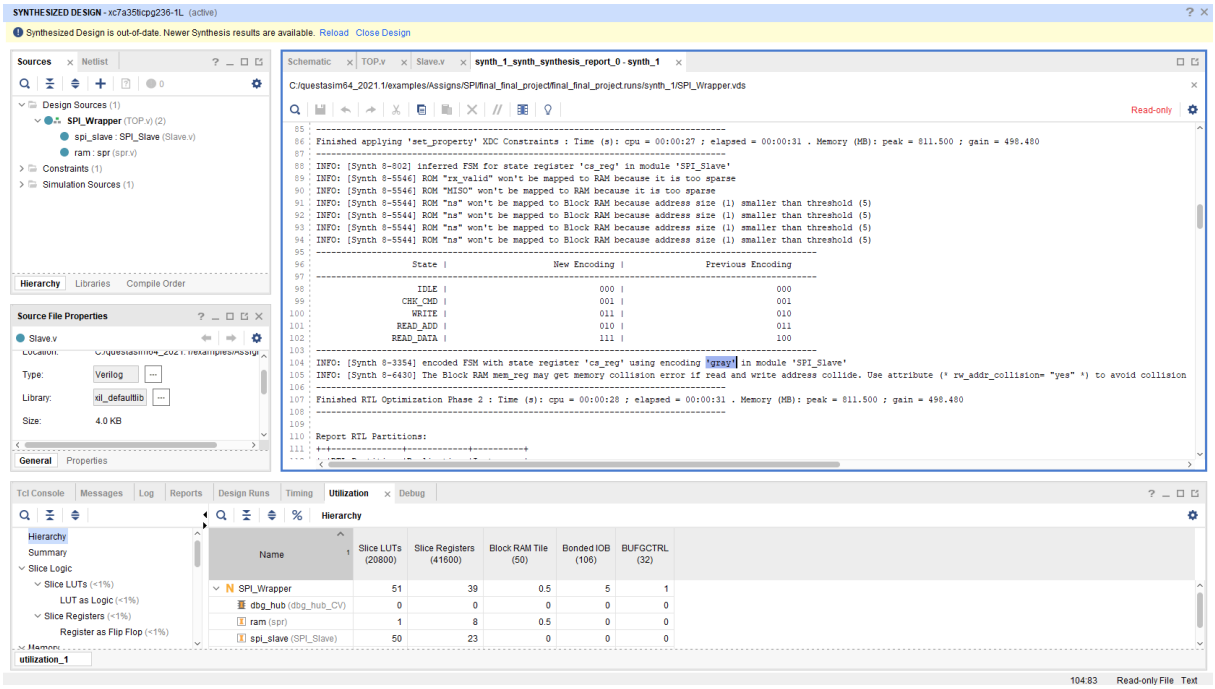


Figure 9.2 — Synthesis report: FSM state register 'cs_reg' encoded with 'gray' in module SPI_Slave, with Utilization by hierarchy

9.1.3 Timing Report

Post-synthesis timing closure is met with a Worst Negative Slack (WNS) of 6.604 ns, Worst Hold Slack (WHS) of 0.160 ns, Worst Pulse Width Slack (WPWS) of 4.500 ns, and zero failing endpoints (97

endpoints for setup/hold, 40 for pulse width). All user-specified timing constraints are met — timing is comparable to the one-hot run (6.657 ns WNS).

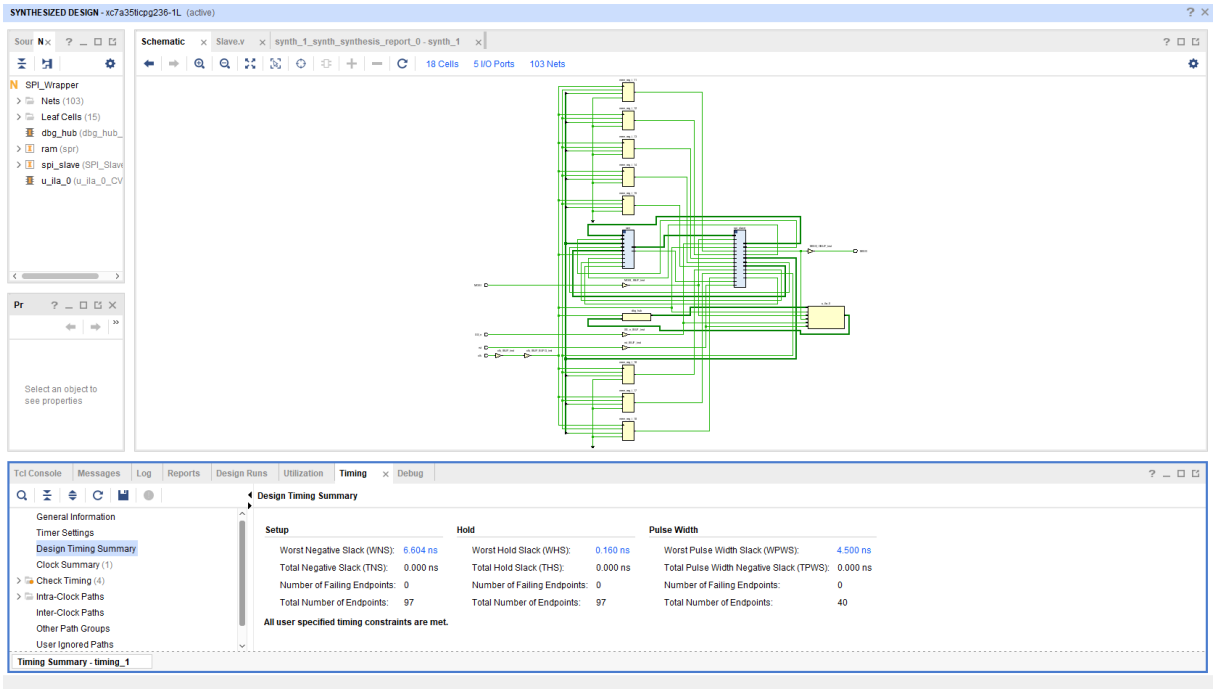


Figure 9.3 — Post-synthesis Design Timing Summary for the Gray-encoded run: WNS 6.604 ns, 0 failing endpoints

9.1.4 Critical Path

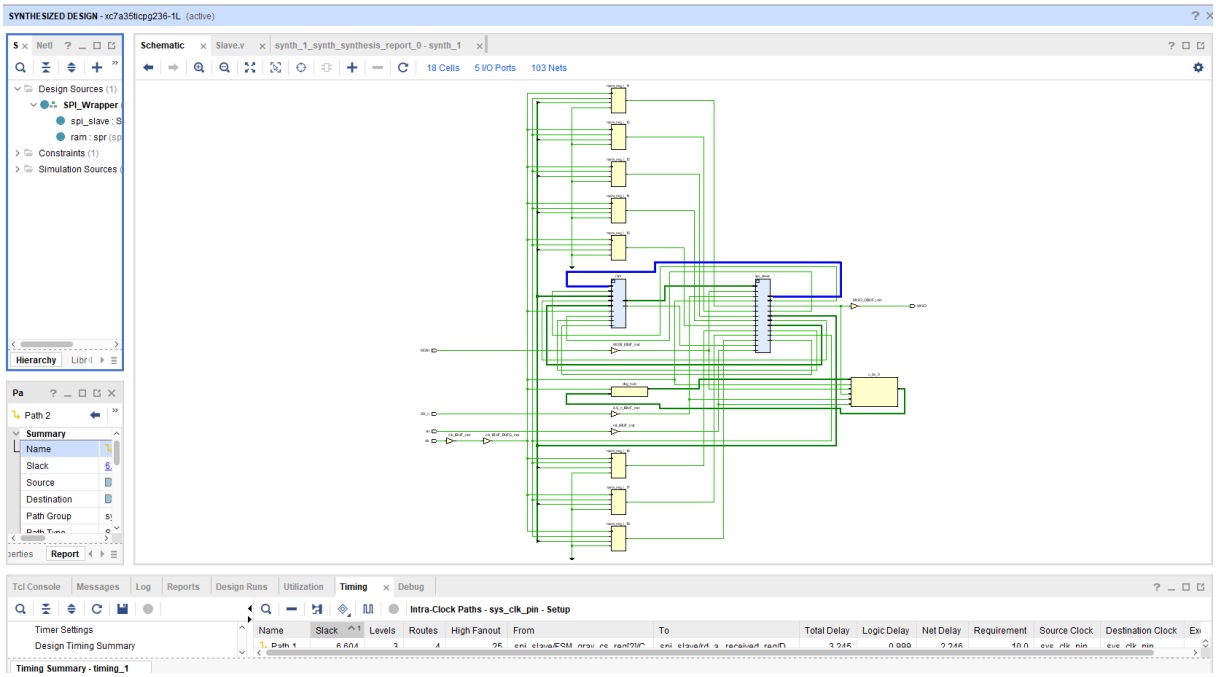


Figure 9.4 — Schematic with the critical path highlighted, and the Intra-Clock Paths setup timing report for Path 2 (slack 6.604 ns)

9.2 Implementation

9.2.1 Utilization Report

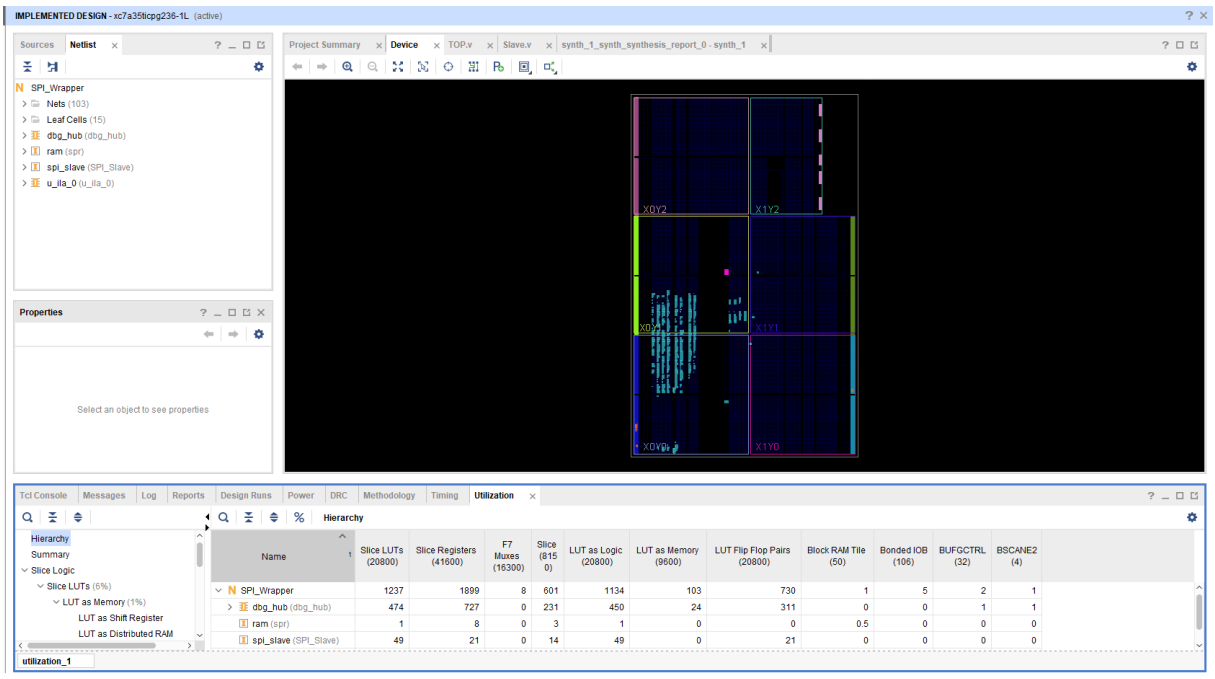


Figure 9.5 — Implemented design device view with post-implementation Utilization by hierarchy, Gray-encoded run

9.2.2 Timing Report

Post-implementation timing closure is met with a Worst Negative Slack (WNS) of 4.503 ns, Worst Hold Slack (WHS) of 0.036 ns, Worst Pulse Width Slack (WPWS) of 3.750 ns, and zero failing endpoints across

setup (3787 endpoints), hold (3771 endpoints), and pulse-width (2076 endpoints) checks. All user-specified timing constraints are met.

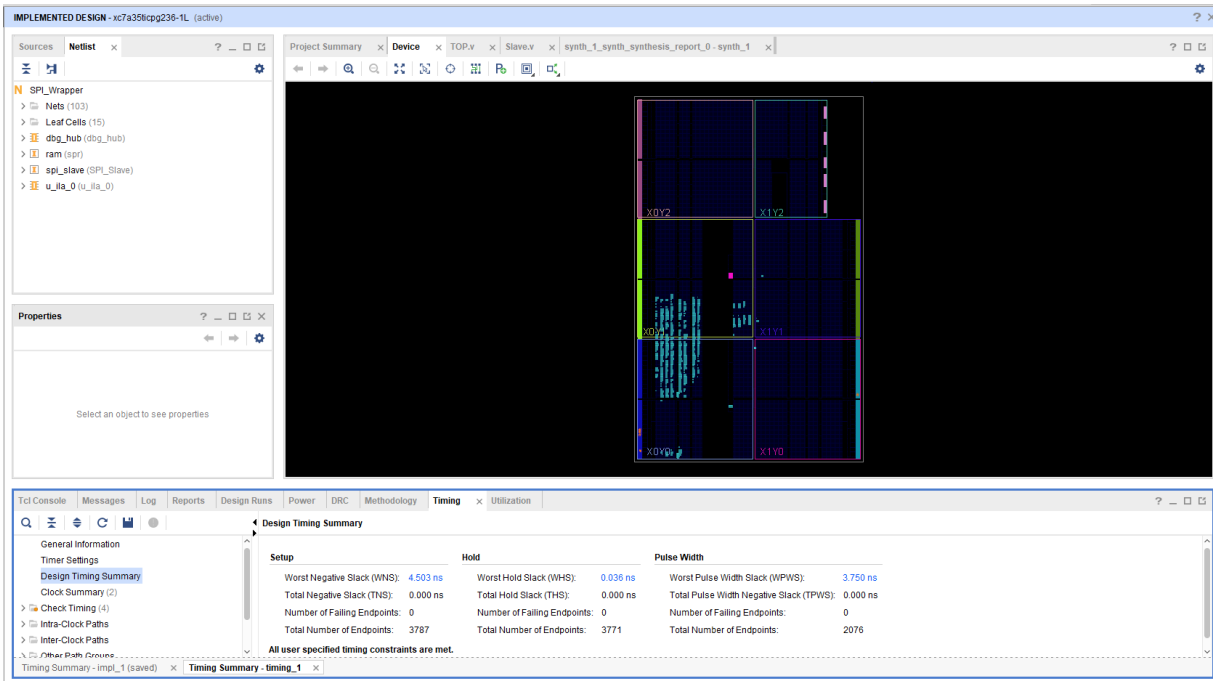


Figure 9.6 — Implemented design device view with post-implementation Design Timing Summary, Gray-encoded run: WNS 4.503 ns, 0 failing endpoints

9.2.3 messages

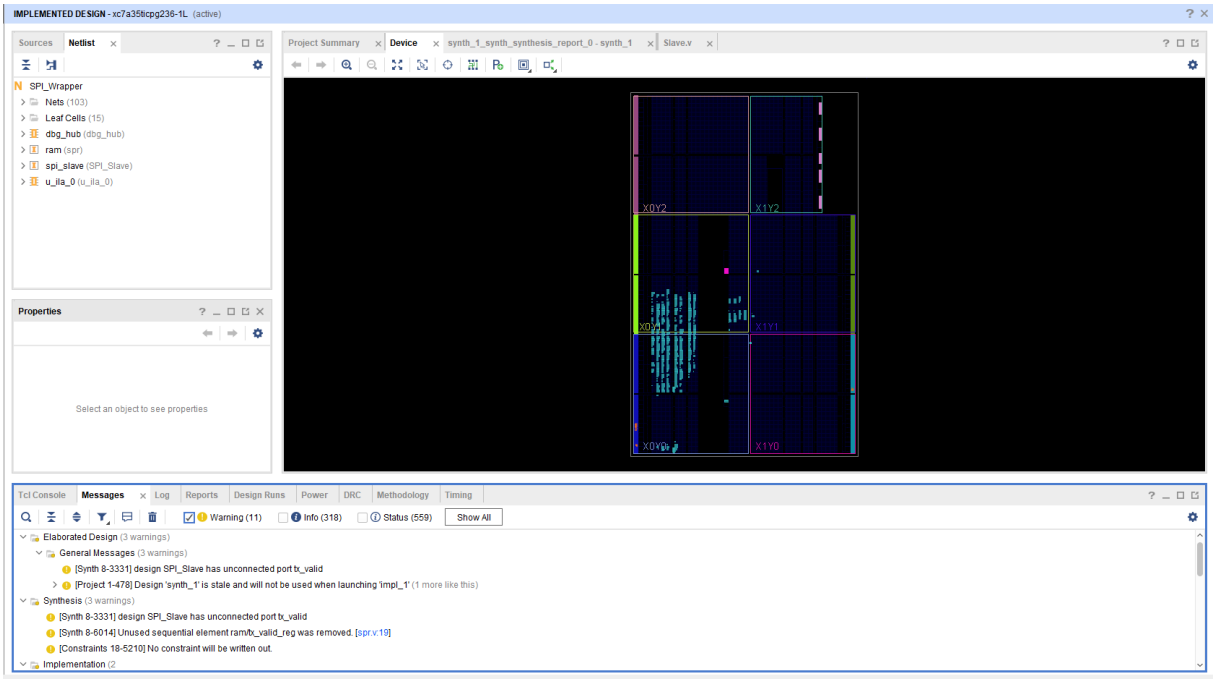


Figure 9.7 — Implemented design device (placement) view on xc7a35ticpg236-1L, Gray-encoded run

10. Encoding: Sequential

This chapter groups the complete Vivado results — synthesis followed by implementation — for the FSM state register (cs_reg) left at Vivado's default sequential (binary) encoding.

10.1 Synthesis

10.1.1 Utilization

With the state register left at Vivado's default 'sequential' encoding, SPI_Wrapper again synthesizes to 18 cells, 5 I/O ports (104 nets — one more than the gray/one-hot runs).

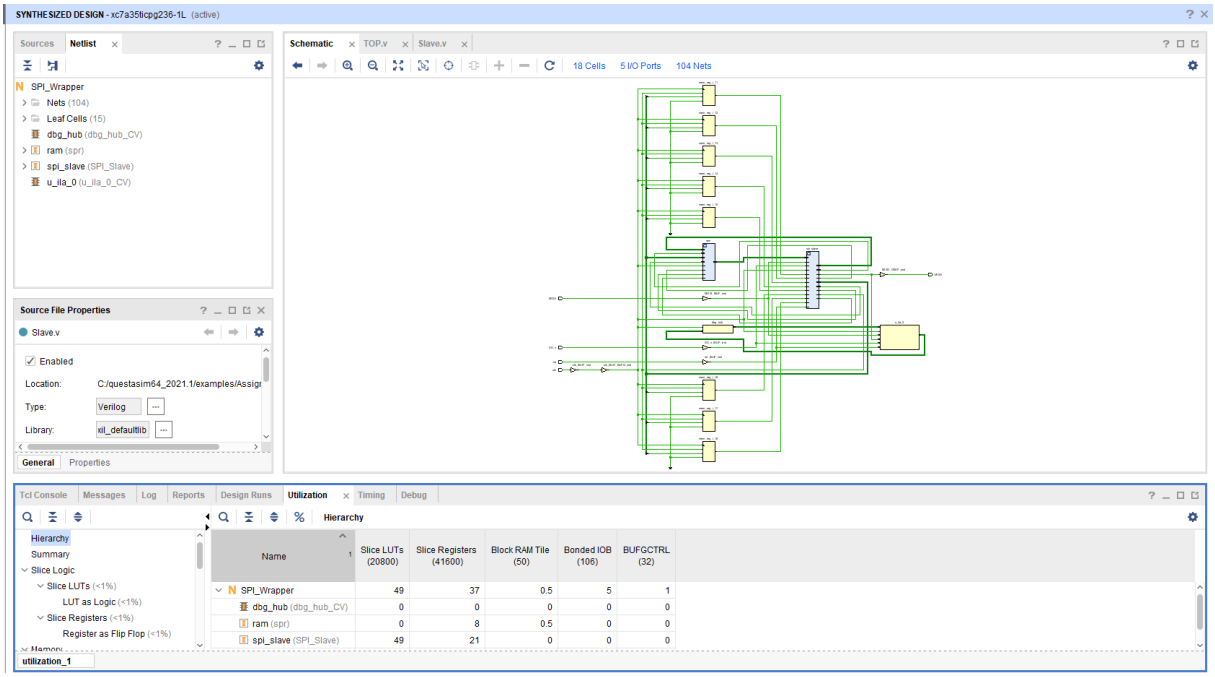


Figure 10.1 — Post-synthesis schematic of SPI_Wrapper for the Sequential-encoded run, with post-synthesis Utilization by hierarchy

10.1.2 Synthesis Report (Encoding Used)

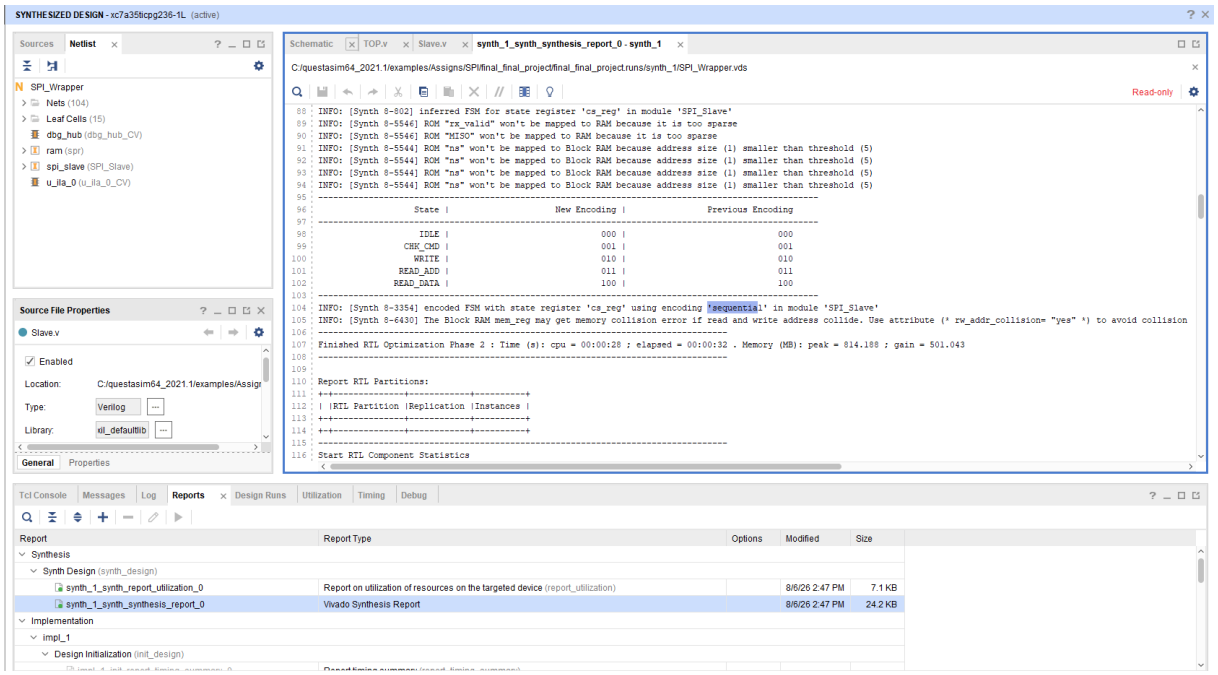


Figure 10.2 — Synthesis report: FSM state register 'cs_reg' encoded with 'sequential' in module SPI_Slave

10.1.3 Timing Report

Post-synthesis timing closure is met with a Worst Negative Slack (WNS) of 6.710 ns, Worst Hold Slack (WHS) of 0.161 ns, Worst Pulse Width Slack (WPWS) of 4.500 ns, and zero failing endpoints (97 endpoints for setup/hold, 40 for pulse width) — the best WNS of the three encodings compared (gray 6.604 ns, one-hot 6.657 ns, sequential 6.710 ns).

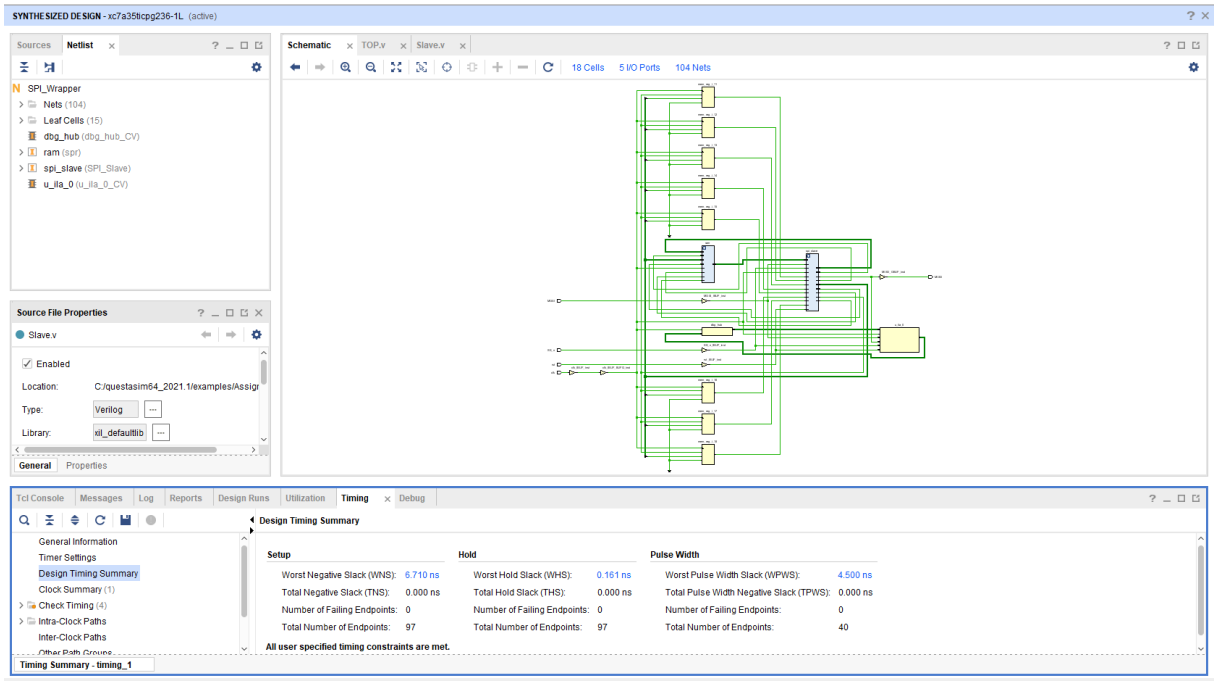


Figure 10.3 — Post-synthesis Design Timing Summary for the Sequential-encoded run: WNS 6.710 ns, 0 failing endpoints

.1.4 Critical Path

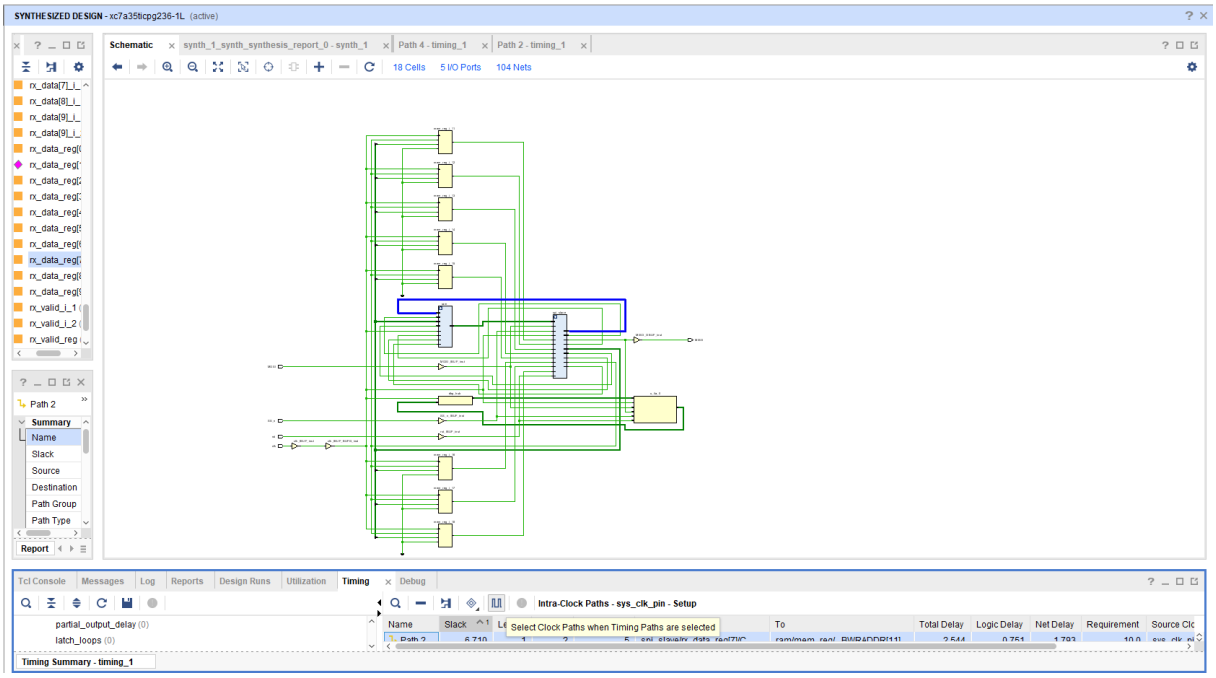


Figure 10.4 — Schematic with the critical path highlighted, and the Intra-Clock Paths setup timing report for Path 2 (slack 6.710 ns)

10.2 Implementation

10.2.1 Utilization Report

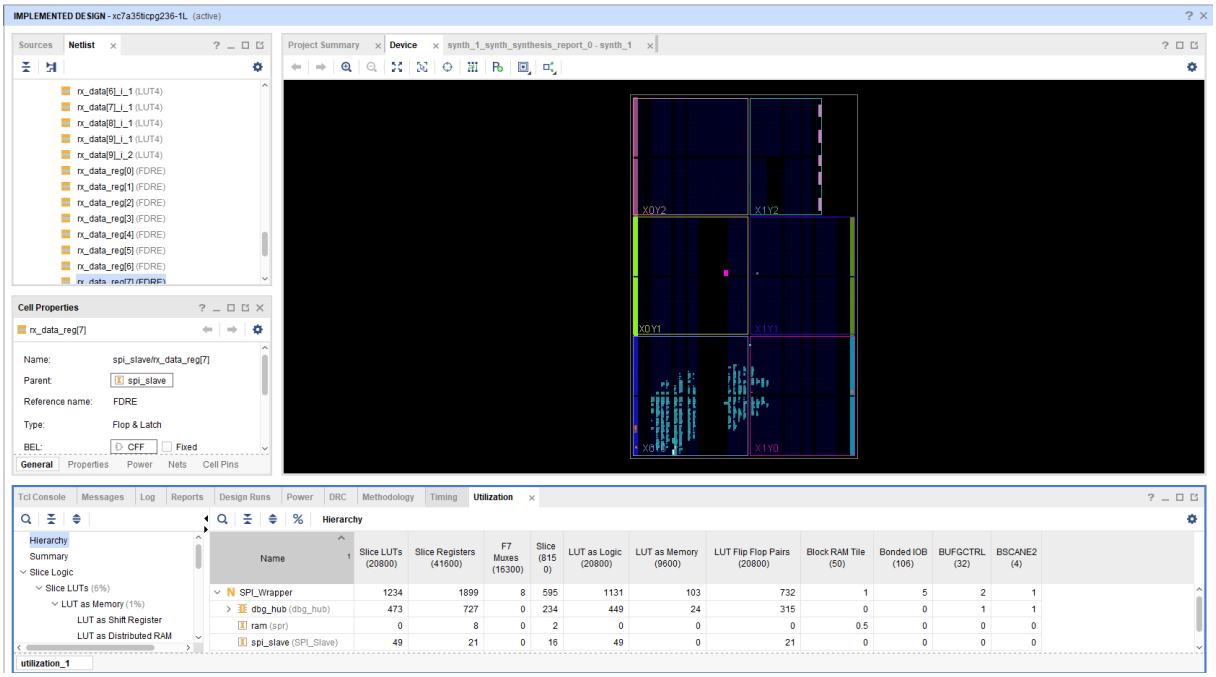


Figure 10.6 — Implemented design device view with post-implementation Utilization by hierarchy, Sequential-encoded run

10.2.2 Timing Report

Post-implementation timing closure is met with a Worst Negative Slack (WNS) of 4.241 ns, Worst Hold Slack (WHS) of 0.048 ns, Worst Pulse Width Slack (WPWS) of 3.750 ns, and zero failing endpoints across setup (3787 endpoints), hold (3771 endpoints), and pulse-width (2076 endpoints) checks. All user-specified timing constraints are met.

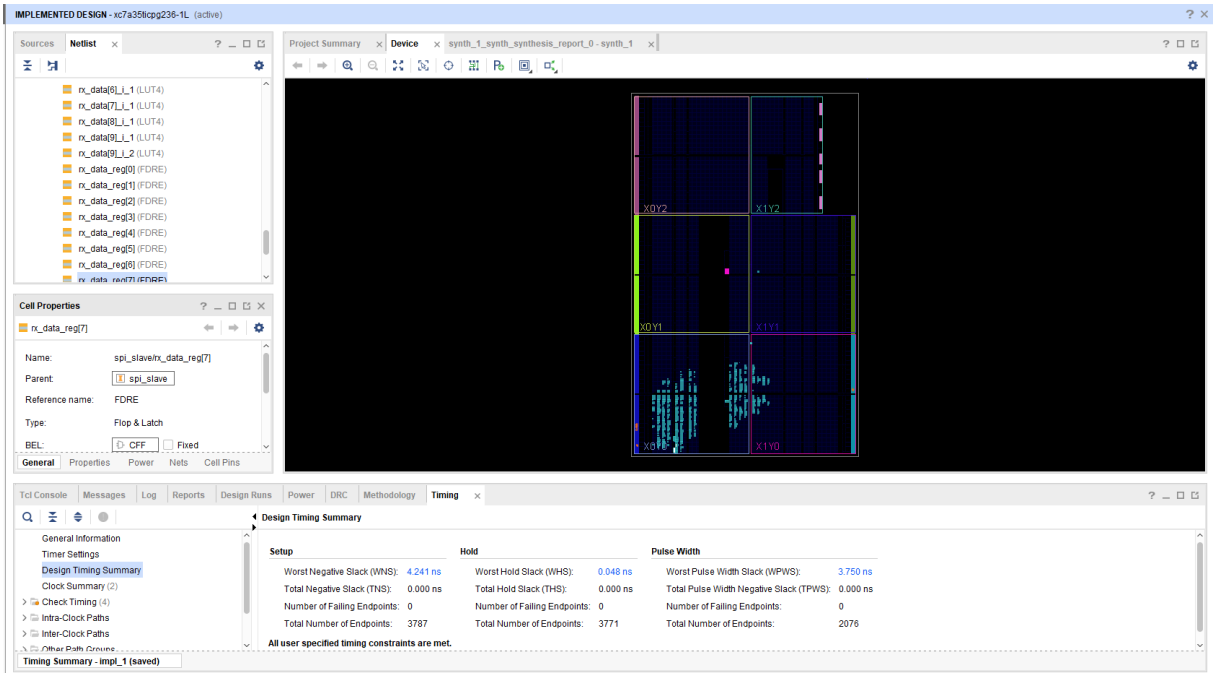


Figure 10.7 — Implemented design device view with post-implementation Design Timing Summary, Sequential-encoded run: WNS 4.241 ns, 0 failing endpoints

10.2.3 Messages

The placed design on the xc7a35ticpg236-1L fabric for the Sequential-encoded run, showing the implemented-design Messages summary (6 Warnings, 266 Info, 515 Status).

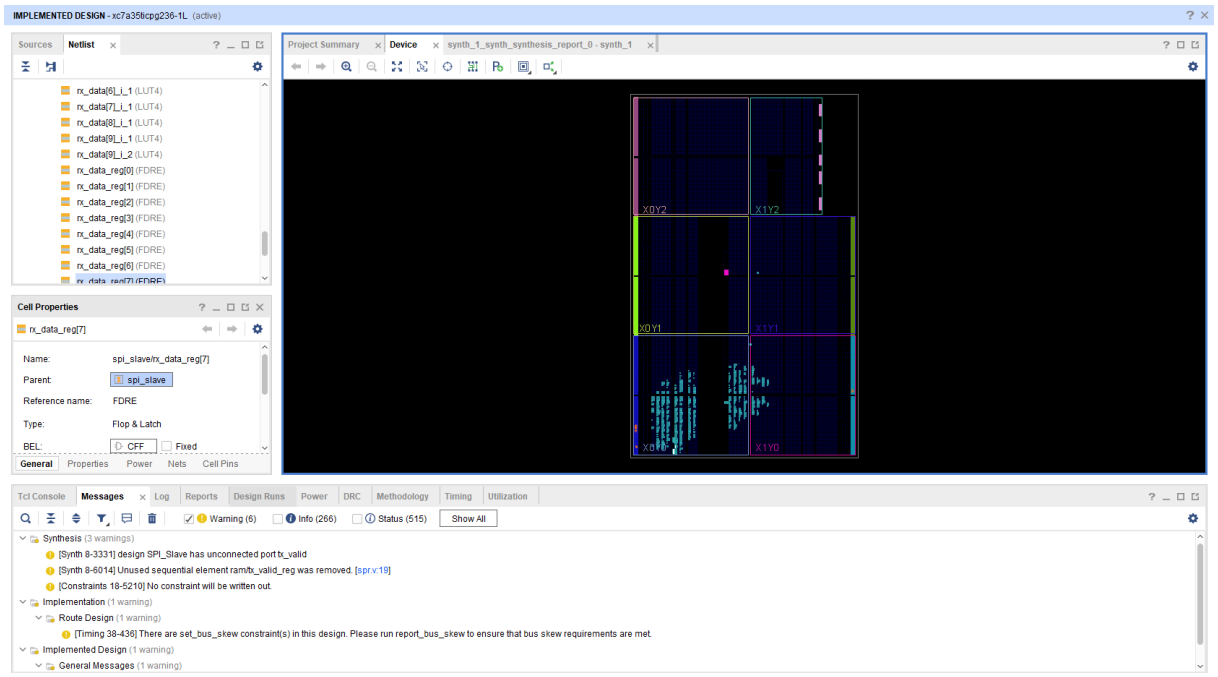


Figure 10.8 — Implemented design device (placement) view on xc7a35ticpg236-1L, Sequential-encoded run

Duplicate Chapter 8/9/10 (with their Synthesis/Implementation subsections) for any additional FSM encoding compared in this project.

11. Encoding Comparison Summary

The table below summarizes post-synthesis and post-implementation results across all three FSM encodings evaluated for the SPI_Slave state register (cs_reg).

Encoding	Synth WNS (ns)	Impl WNS (ns)	Impl LUTs	Impl Registers	Failing Endpoints
One-hot	6.657	4.215	1240	1901	0
Gray	6.604	4.503	1237	1899	0
Sequential	6.710	4.241	1234	1899	0

All three encodings meet timing with zero failing endpoints and produce nearly identical resource utilization (within 6 LUTs / 2 registers of each other), confirming that for this design's size, FSM state encoding has minimal impact on area or timing — the differences fall within normal place-and-route variance rather than reflecting a meaningful trade-off between encodings.

12. Recommended Encoding

Comparing the three encodings on the metrics that matter for a real, routed design: Gray encoding gives the best post-implementation timing margin (WNS 4.503 ns, noticeably ahead of Sequential's 4.241 ns and One-hot's 4.215 ns), while Sequential encoding gives the smallest resource footprint (1234 LUTs, the

fewest of the three, though registers are tied with Gray at 1899). One-hot encoding — despite commonly being favored for speed in larger FSMs — shows no advantage here: it uses the most LUTs/registers of the three and has the tightest post-implementation timing margin.